

基于Harness Engineering架构的企业实战项目

讲师：肖斌

注意：该项目已经讲了4天了还有3天



第一、项目代码架构图

代码块

```
1  src/
2  |   agent/                                # Agent 核心
3  |   |   main_agent.py                    # 主 Agent 入口 (11阶段初始化流水线)
4  |   |   config.py                        # 全局配置 (模型/沙箱/MongoDB/路
      径/Store)
5  |   |   schema.py                        # 数据结构 (ProcurementContext / SSE事件
      模型 / API模型)
6  |   |   env_utils.py                    # 环境变量读取
7  |   |   middleware_config.py            # 子 Agent 中间件工厂
8  |   |
9  |   |   memory/
10 |   |   |   AGENTS.md                    # 主 Agent 通用行为准则 (沙箱
      /AGENTS.md)
11 |   |   |   prompts.py                  # 主 Agent system_prompt (精简版协调者角
      色)
12 |   |
13 |   |   backends/
14 |   |   |   sandbox_setup.py            # 沙箱创建 + 基础文件播种 (seed)
15 |   |   |   custom_opensandbox.py      # OpenSandboxBackend 自定义封装
16 |   |
17 |   |   middlewares/
18 |   |   |   context_injection.py        # 1. 注入用户上下文到 system prompt
19 |   |   |   skills_sync.py              # 2. 本地技能 → 沙箱同步 (增量)
20 |   |   |   user_skills_restore.py      # 3. StoreBackend → 沙箱恢复持久化技能
21 |   |   |   tools_summarization.py      # 4. SummarizationToolMiddleware 封装
22 |   |   |   memory_update.py            # 5. 自动更新 recent_suppliers /
      recent_queries
23 |   |
24 |   |   subagents/
```

```

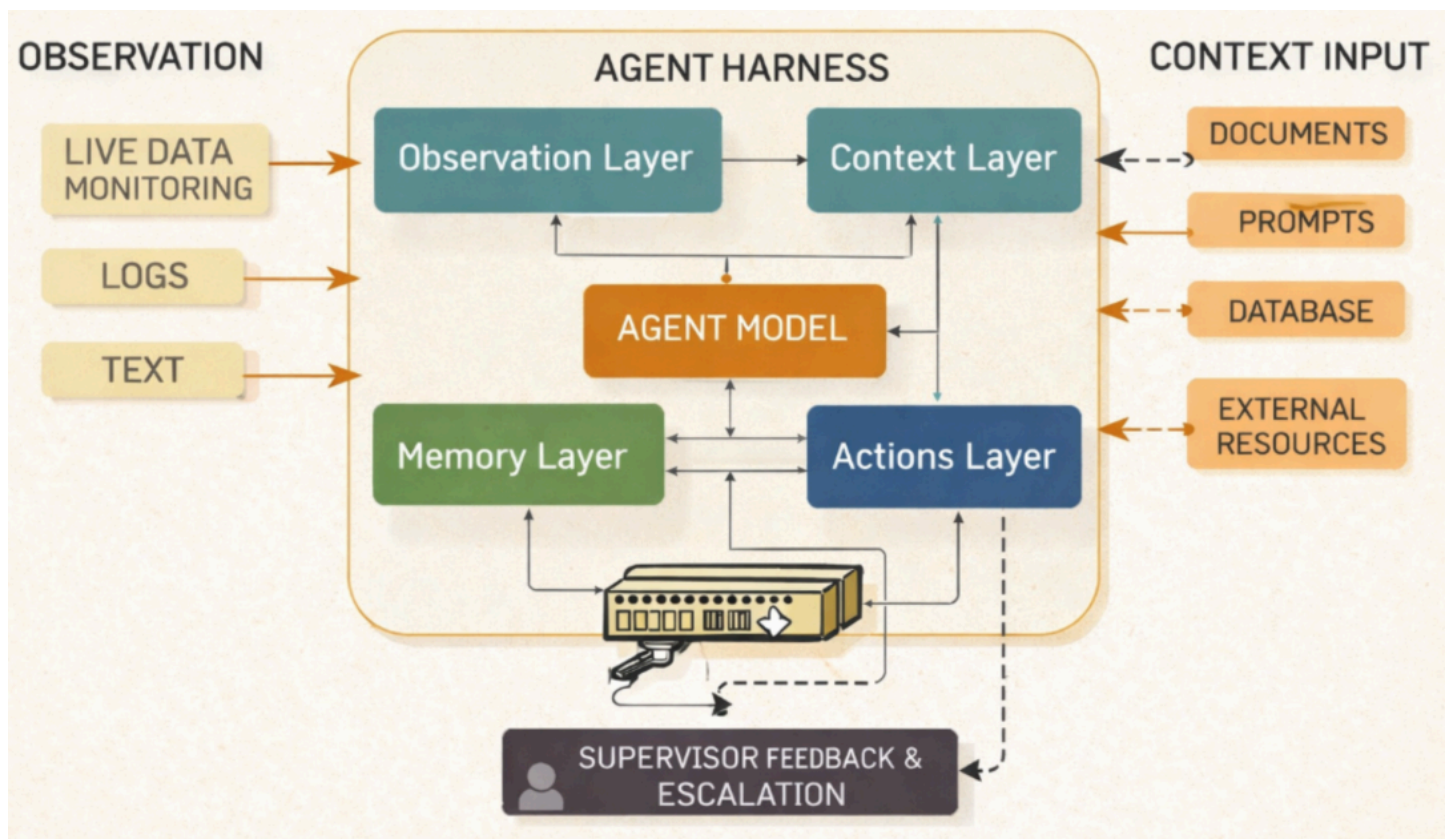
25 | | | └─ loader.py # YAML 配置加载 + 工具名称匹配解析
26 | | | └─ configs/
27 | | |   └─ procurement_analyst.yaml # 分析子 Agent 配置
28 | | |   └─ procurement_order.yaml # 订单子 Agent 配置
29 | | |
30 | | └─ tools/
31 | |   └─ mcp_client.py # MCP 多服务器连接 + 工具按前缀分组
32 | |   └─ chart_generator.py # 26种图→1个 generate_visualization 入
    □
33 | |   └─ web_search.py # 智谱搜狗 Web 搜索
34 | |   └─ hitl_tools.py # request_order_info (数据补充中断)
35 | |   └─ assign_skill.py # 技能分配 (复制+StoreBackend持久化+清
    理)
36 | |   └─ download_sandbox_file.py # 沙箱文件下载到本地
37 | |
38 | └─ skills/ # 技能资源 (本地 → 沙箱同步)
39 |   └─ main/skill-management/ # 技能生命周期管理
40 |   └─ procurement/
41 |     └─ chart_params.md # 26种图表完整参数参考
42 |     └─ procurement-analysis/SKILL.md # 分析流程操作手册 (5步)
43 |     └─ supplier-price-urls/ # 供应商报价URL映射表
44 |     └─ web-scraper/ # 网页抓取 (直接HTTP+HTML→MD)
45 |
46 | └─ api_view/ # FastAPI Web层
47 |   └─ web_main.py # FastAPI app 入口 (lifespan/路由/CORS)
48 |   └─ agent_loader.py # Agent 懒加载 + MongoDB 管理
49 |   └─ web_config.py # API 元信息
50 |   └─ api/
51 |     └─ chat.py # SSE 流式对话 + 中断检测 + resume 端点
52 |     └─ history.py # 历史会话管理 (MongoDB CRUD)
53 |
54 | └─ mcp_server/ # MCP Server (ERP 业务接口代理)
55 |   └─ server_main.py # MCP 服务入口 (端口8000)
56 |   └─ tools/
57 |     └─ suppliers_tools.py # supplier_query / supplier_*
58 |     └─ parts_tools.py # part_query / part_search /
    part_by_supplier
59 |   └─ order_tools.py # order_create / order_update /
    order_search_details
60 |     └─ inventory_tools.py # inventory_warning
61 |
62 | └─ test/
63 |   └─ agent_test.py # 终端测试脚本

```

第二、项目中具体Harness架构的实现

1、什么是 Harness Engineering?

- **Prompt Engineering** (2022-2024 主流) 主要优化单次交互的质量, 重点是 “一句提示词该怎么写, 模型才更容易给出想要的结果”
- **Context Engineering** (2025 年2月后兴起) 则动态构建知识、记忆、RAG, 解决 “模型看什么” 的问题, 减少幻觉、提高检索命中率
- **Harness Engineering** (当前阶段) 重点构建整个运行环境, 解决 “模型怎么把长链路任务稳定做完” 的系统级问题。



Harness Engineering 是当前 AI Agent 开发领域的一个核心范式转变。它不再仅仅关注如何通过提示词 (Prompt Engineering) 或上下文管理 (Context Engineering) 来 “优化模型输出”, 而是聚焦于 **构建包裹在模型之外的一整套系统化基础设施**, 旨在将大语言模型 (LLM) 那种 “不稳定、非确定性” 的智能, 转化为能够 **稳定、可靠、长时间执行复杂任务** 的工作引擎。

- **核心定义:** Harness (马具/缰绳) 指的是除了模型本身之外的所有东西——工具、记忆、规划、安全护栏、执行循环、状态管理等。LangChain 团队将其精炼为公式: **Agent = Model + Harness**。模型提供 “思考” 能力, 而 Harness 负责 “让它真的能干活, 而且别出大事”。
- **解决的问题:** 传统 Agent 在应对长周期、多步骤的复杂任务时, 常面临上下文窗口爆炸、状态丢失、工具调用混乱、缺乏规划、无法从失败中恢复等工程难题。Harness Engineering 正是为了解决这些 “工程上的 ‘稳’ 的问题” 而生的。

一个完整的Agent Harness由几个相互配合的核心能力层构成，它们共同决定了Agent的行为和性能：

- 任务规划：通过内置的任务清单功能，将复杂目标分解为一系列可执行的子任务，并追踪每一步的执行状态。
- 虚拟文件系统：为Agent提供一个内置的虚拟文件管理系统，支持读写、编辑、搜索等操作，使其能像人类开发者一样进行工作。
- 子Agent委派：主Agent可以创建专门的“子Agent”来执行特定任务，实现任务隔离和上下文压缩。
- 上下文管理：通过文件系统等策略，管理长对话中的Token消耗，防止因上下文溢出而导致性能下降。
- 可插拔中间件：通过一组Hook函数，在模型和工具调用的关键节点注入自定义逻辑，实现缓存、重试、日志、PII检测等增强功能。
- Skills技能系统：将提示词、文档、脚本等打包为可复用的标准模块，为Agent注入特定领域的知识和能力。并做到自我进化。
- 持久化记忆：为Agent提供短期（对话历史）和长期记忆能力，使其能从历史交互中学习。

2、项目中Herness架构的核心能力(一)：长周期复杂任务的Planning能力

提供 `write_todos` 工具，Agent 可维护结构化任务列表（pending/in_progress/completed），持久化在 Agent state 中。

项目实现：

项目	说明
DeepAgents 内置	<code>write_todos</code> 工具由框架自动注册，无需显式配置
受益方	三个 Agent 全部受益：主 Agent 协调复杂任务、analyst 执行 5 步 order 执行 4 步订单操作
典型触发	analyst 启动时："ls → 读 SKILL.md → 收集数据 → 分析 → 图表 → 自动建议拆为 todo
持久化	<code>CHECKPOINTER</code> (MongoDBSaver) 持久化 Agent state，跨重启



TOOLwrite_todosa3b92224-3090-a8ef-d6d4-80c9cb72d8c5完成

▶ 参数

▼ 结果

任务清单

Updated todo list to [{ 'content': 'ERP内部数据收集：查火花塞零件信息和供应商', 'status': 'completed' }, { 'content': '外部数据采集：查URL映射、抓取报价页面或web搜索', 'status': 'in_progress' }, { 'content': '执行分析：价格、性能参数、性价比多维度对比', 'status': 'pending' }, { 'content': '生成图表（bar为主，辅以radar/waterfall等）', 'status': 'pending' }, { 'content': '生成最终报告 /analysis/report_{timestamp}.md', 'status': 'pending' }]

3、项目中Herness架构的核心能力(二)：安全的和动态路由的文件系统

项目实现：通过 CompositeBackend 实现三层路由文件系统。

路由分流效果：

路径前缀	后端	生命周期	隔离粒度
/AGENTS.md	OpenSandbox	沙箱生命周期	全局一份（只读）
/memories/{user_id}/	StoreBackend → MongoDB	跨会话永久	按 user_id 隔离
/persisted-skills/	StoreBackend → MongoDB	跨会话永久	按 namespace 组织
/skills/	OpenSandbox	沙箱生命周期	每次沙箱重建
/data/、/analysis/	OpenSandbox	沙箱生命周期	临时工作区

代码块

```
1 backend = lambda rt: CompositeBackend(  
2     default=sandbox_backend,          # OpenSandbox（临时文件、代码执行）  
3     routes={  
4         "/memories/": StoreBackend(    # 用户偏好 → MongoDB 持久化  
5             runtime=rt,  
6             namespace=lambda rt: (getattr(rt.runtime.context, 'user_id',  
7 'default_user')),),  
8         "/persisted-skills/": StoreBackend( # 持久化技能 → MongoDB  
9             runtime=rt,  
10            namespace=lambda rt: SKILLS_STORE_NAMESPACE,  
11        ),  
12    },  
13 )
```

4、项目中Herness架构的核心能力(三)： Subagent的任务委派

主 Agent 拥有 `task` 工具，每次调用创建全新 Agent 实例，独立上下文，执行完返回单个报告。支持并行执行和特殊化配置。

项目实施：

子 Agent 配置加载：loader.py

代码块

```
1  YAML 配置文件 (configs/*.yaml)
2  | load_subagent_configs()      ← YAML → dict 列表
3  | resolve_subagent_tools()    ← 工具名子串匹配 → 可调用对象
4  | _validate_subagent_config() ← 必填字段校验
5  ▼
```

两个子 Agent 对比：

	procurement-analyst	procurement-c
YAML	procurement_analyst.yaml	procurement_order.yam
description	含触发关键词 ("分析"、"对比"、"报告"...)	含触发关键词 ("下单"、"单"、"修改"...)
工具数	8 个 (6 MCP + web_search + generate_visualization)	5 个 (3 MCP + web_search request_order_info)
技能 scope	/skills/procurement/	/skills/order/
中间件	Summarization + 调用限制(50/200)	调用限制(20/50)
interrupt_on	无	order_create: approve/reorder_update: approve/r
context 隔离	独立上下文，analyst 长篇工具结果不污染主 Agent	独立上下文，订单操作不干

委派模板（AGENTS.md:74-121）：

主 Agent 的 `AGENTS.md` 为每个子 Agent 定义了结构化的 `task` 调用模板，确保 user_id/username/偏好 正确传递到子 Agent。

5、项目中Herness架构的核心能力(四)：Context Management — 上下文管理

四种策略——Input context（启动时加载）、Compression（自动 offload + summarization）、Isolation（子 Agent 隔离）、Long-term memory（跨线程持久化）。

项目实施：

a. Input Context

输入	来源	何时加载
system_prompt	prompts.py	Agent 创建时
AGENTS.md	本地 → 沙箱 /AGENTS.md (main_agent.py:122-124)	Agent 创建时，memory 持久化
用户偏好	/memories/{user_id}/preferences.md (StoreBackend)	每轮对话前 read_file
技能 frontmatter	各 SKILL.md 的 YAML frontmatter (渐进式披露)	Agent 启动时扫描
用户上下文	ContextInjectionMiddleware 注入 SystemMessage	每轮 before_agent
子Agent报告路径/订单确认	委派结果	收到返回后

b. Compression

机制	实现	配置项
自动 Offloading	DeepAgents 内置，工具结果 > 20k tokens → 写入文件，上下文只保留路径 +前10行预览	框架默认
自动 Summarization	SummarizationMiddleware，上下文 85% 窗口 → 自动压缩历史为摘要，完整历史保存到 /conversation_history/	tools_summarization.py
手动 compact_conversation	SummarizationToolMiddleware 注册的工具，Agent 在收到子Agent长篇报告/对话超6轮/用户频繁切换话题时主动调用	tools_summarization.py model=deepseek

c. Isolation（子 Agent 上下文隔离）

analyst 执行 5 步分析流程时会产生大量中间结果（MCP 返回的供应商/零部件 JSON、Python 脚本输出、图表生成参数等），全部隔离在子 Agent 自己的上下文窗口中。主 Agent 只收到最终的结构化报告。

d. Long-term Memory


```
25 | → store.aput() 持久化到 StoreBackend |
26 | → UserSkillsRestoreMiddleware 恢复（下次沙箱创建时） |
27 | → 清理压缩包 |
28 |_____|
```

基于SkillsSyncMiddleware发现链

代码块

```
1  yaml system_prompt: "ls /skills/procurement/ 扫描可用技能"
2    → 返回: chart_params.md, procurement-analysis/, supplier-price-urls/, web-
   scraper/
3    → read_file("procurement-analysis/SKILL.md") 加载操作手册
4    → 按流程激活各技能
```

7、项目中Herness架构的核心能力(六): Human-in-the-Loop — 人工介入

通过 `interrupt_on` 参数选择性地在工具调用前暂停，等待人工审批或修改。

项目实施：双层中断体系。

代码块

```
1  第1层：数据补充 (tool-internal interrupt)
2    request_order_info 工具内部调用 interrupt()
3    └─ 触发时机：Schema校验发现必填字段缺失
4    └─ 恢复格式：{"supplement": "自由文本补充信息"}
5    └─ 循环：校验→补充→解析→校验...直到完整
6
7  第2层：最终审批 (interrupt_on 配置)
8    order_create / order_update 的 interrupt_on 配置
9    └─ 触发时机：数据完整，准备执行
10   └─ 恢复格式：{"decisions": [{"type": "approve"|"reject"}]}
11   └─ 特性：approve → 执行 / reject → 取消
```

代码位置：


```
5  【自动路径】对话结束后
6    → MemoryUpdateMiddleware.aafter_agent()
7    → LLM 提取 {suppliers: [...], query: "..."}
8    → store.aput() 写入 StoreBackend
9    → 跨会话保留
```

9、项目的最大安全问题： 基于OpenSandbox来解决

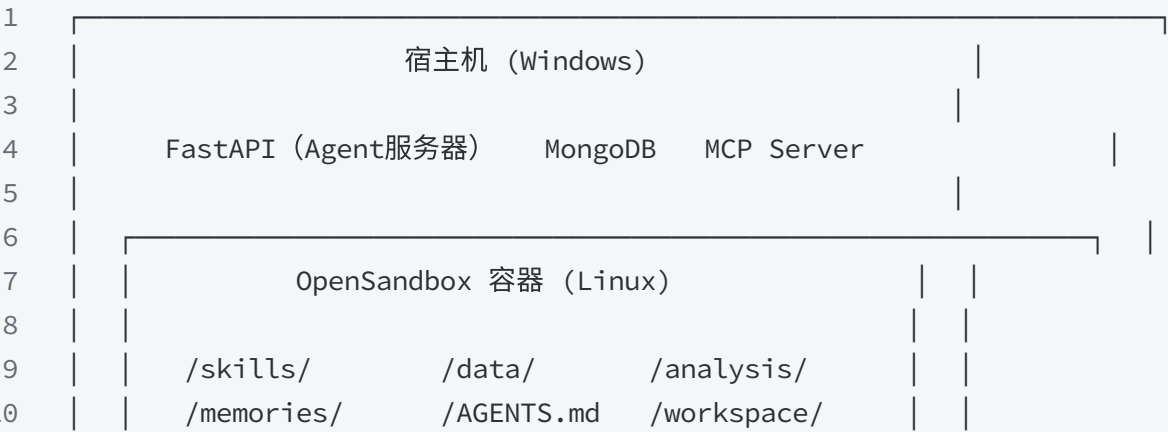
OpenSandbox 就是这个项目的**唯一安全边界**。所有 Agent 行为——运行 Python 脚本、执行 Shell 命令、读写文件、下载技能、爬取网页——全部发生在这个容器内部。OpenSandbox 的配置决定了这道墙有多厚。

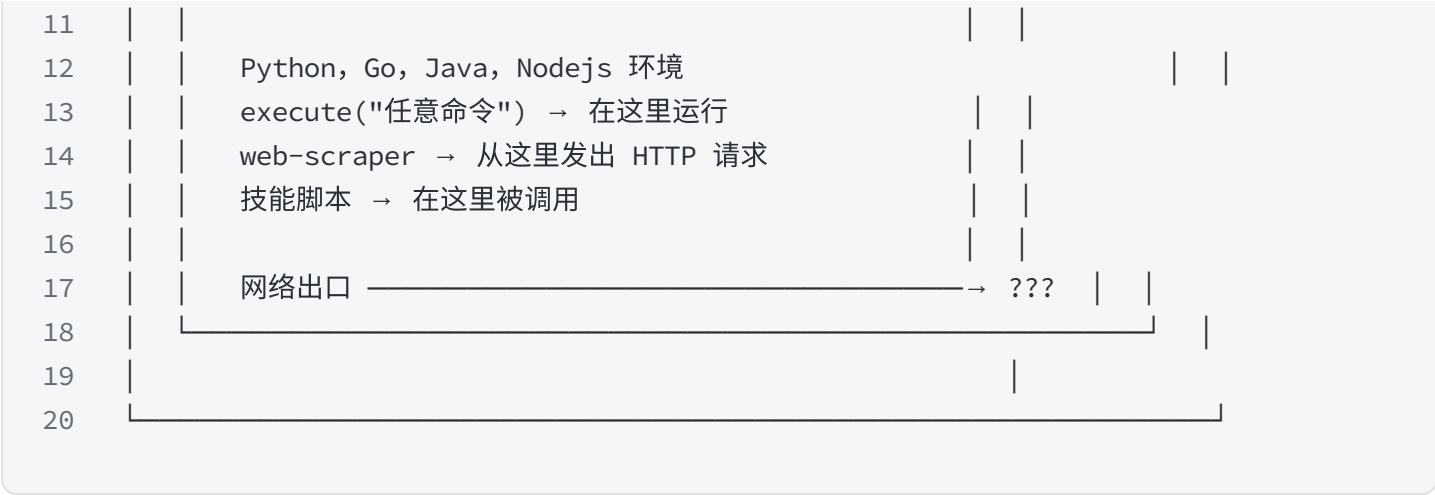
综合风险评估

风险	等级	影响	当前状态
沙箱无网络出口限制	高	数据外传、内网探测、云凭证窃取	network_policy 解决
无 URL 白名单 (download_skill)	中	从恶意服务器下载	network_policy 解决
skill_name 路径穿越	中	assign_skill 访问任意沙箱路径	文件操作
user_id 隔离弱	中	多用户数据可被跨用户访问	文件操作
AGENTS.md 无技术只读	低	Agent 可能修改全局准则	防止文件修改
无边载配额/进程限制	低	沙箱资源耗尽	docker容器可以随时增
download_sandbox_file 无敏感文件过滤	低	沙箱文件可外传到本地	无限制

Sandbox 做边界定义：墙内 vs 墙外

代码块





OpenSandbox 防护了三件事：

防护	机制
宿主机文件系统不可见	容器内 /skills/、/data/ 都是容器自己的文件系统 宿主机 E:\my_project\ 物理隔离
进程隔离	execute() 启动的进程在容器内运行，无法直接 主机进程
资源限制	cpu: "2"、memory: "4Gi" 防止容器耗尽宿主机